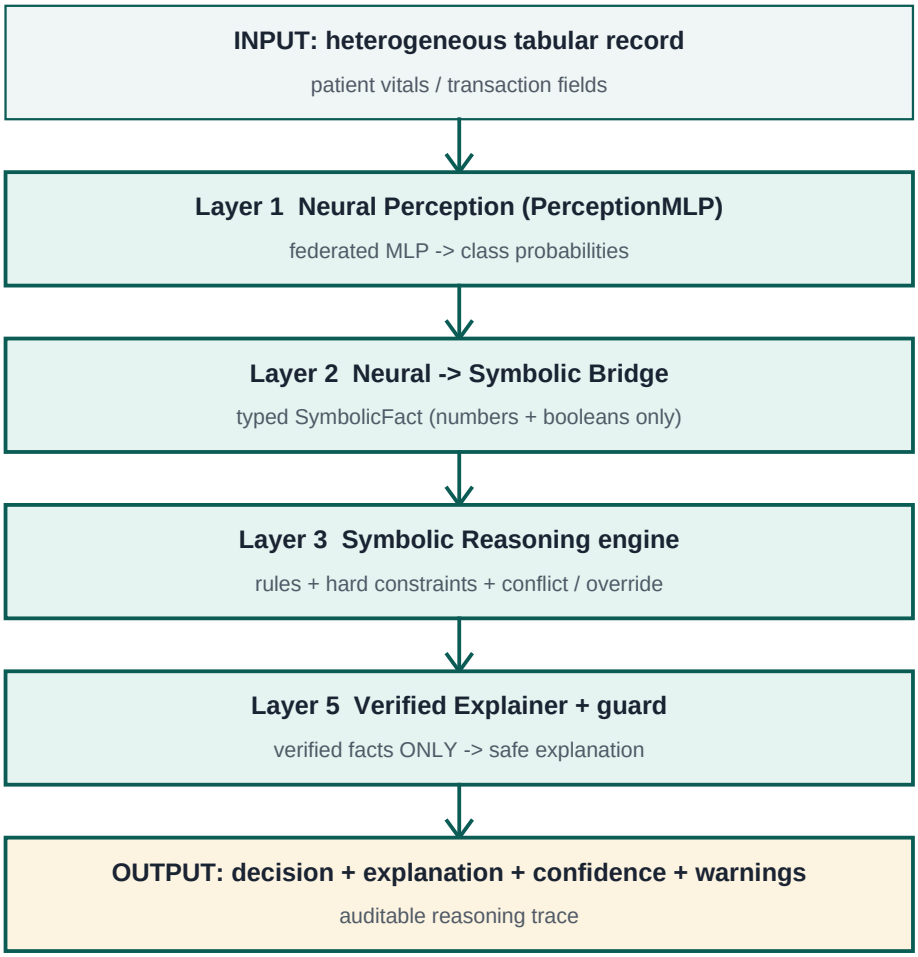
Framework architecture, workflow, LLM usage and explainable AI

Domain-agnostic federated neuro-symbolic framework for reliable, explainable reasoning over privacy-preserving heterogeneous data (Healthcare risk and Banking fraud)

1. What the framework is

The framework answers a single question reliably: given one tabular record, what is the decision, and can that decision be trusted and explained? It does this by chaining a neural model (for accuracy), federated learning (for privacy), a symbolic rule engine (for verifiable logic), and a fact-locked explainer (for safe, human-readable reasoning). It is domain-agnostic: the same pipeline serves Healthcare disease-risk and Banking fraud by swapping only the inputs and the domain rule set.

2. Architecture at a glance



Layer 4 Federated Learning (FedAvg / FedProx / FedAdagrad) trains Layer 1 across clients; raw data never leaves a client.

Data flows top to bottom. Layer 4 (federated learning) is drawn to the side because it is not a step in the per-record path - it is the training process that produces the Layer 1 model beforehand.

3. The layers in detail

Layer	Input	Output	Code
1 Neural Perception	Standardised feature vector	Class probabilities + predicted class	nsf/neural/models.py

Layer	Input	Output	Code
2 Bridge	Prediction + raw features	Typed SymbolicFact (no free text)	nsf/bridge/neural_symbolic.py
3 Symbolic Reasoning	SymbolicFact	Final class, score, fired rules, conflict/override	nsf/symbolic/engine.py + rules.py
4 Federated Learning	Client datasets (local)	Trained global Layer-1 model	nsf/federated/*
5 Verified Explainer	Verified facts only	Explanation + reasoning trace + guard result	nsf/llm/explainer.py

4. How it works: a single record, step by step

1. The record is standardised using the training mean/std, then passed through the federated MLP, which outputs class probabilities (nsf/pipeline.py, infer_and_verify).

2. The Bridge packages the prediction, its confidence and the numeric/boolean features into a typed SymbolicFact. No narrative text is allowed into the logic layer.

3. The Symbolic Reasoner runs every domain rule, sums the points of the rules that fire, and maps the total to a symbolic class.

4. It compares the symbolic class with the neural class. If they agree, that is the decision. If they disagree, the neural class is kept UNLESS a hard safety constraint is violated or neural confidence is below 0.55, in which case the symbolic class overrides it and a warning is recorded.

5. The Verified Explainer receives ONLY the verified-facts package and writes an explanation that can mention just the rules that actually fired. A hallucination guard checks the text; if any drift is found, it falls back to a strict template.

6. The output is the final decision, the confidence, the fired rules, any warnings, and a full reasoning trace saved to results/*_traces.json.

5. How training works (Layer 4)

1. The server sends the current global model to all clients.

2. Each client trains locally on its own private data for a few epochs (SGD, momentum 0.9) and returns only its updated parameters - never the data (nsf/federated/client.py).

3. The server aggregates the client parameters using the chosen strategy (FedAvg size-weighted average, FedProx same aggregation with a client-side proximal term, or FedAdagrad adaptive server update).

4. This repeats for 12 global rounds. Communication cost and accuracy are logged each round.

5. The best-performing global model per domain is saved to models/ and later reused by inference and the dashboard.

Because only parameters move, raw client records stay local - this is the privacy-preserving property. Non-IID client splits (Dirichlet alpha 0.6-0.7) make the comparison of the three strategies meaningful.

6. Is an LLM used? (honest answer)

No external large language model (such as GPT, Claude, Gemini or Llama) and no API key are used in the current system. The component named VerifiedLLMExplainer is a constrained, template-based natural-language generator, not a neural language model. It builds the explanation deterministically from the verified-facts dictionary.

- Why it is built this way: reliability and reproducibility. A free generative model can hallucinate; a template over verified facts cannot invent clinical or financial claims, and it runs offline with no cost or key.
- The 'LLM' naming reflects the intended extension point: an external LLM can be plugged in LATER as a

fact-locked rephraser that only rewords the same verified payload for fluency. It would be given the verified facts as its only allowed content and its output would pass through the same hallucination guard.

- So the architecture is LLM-ready, but the trustworthiness does not depend on an LLM - it depends on the symbolic verification and the guard.

7. The anti-hallucination mechanism

- Verified-facts-only interface: the explainer receives verified_facts_only() - the final label, confidence, fired rules, evidence sentences and flags - and nothing else.
- Fired-rules whitelist: the text may reference only rules that actually fired; naming a rule that scored zero is treated as a violation.
- Entity grounding: the explanation must mention the correct entity id (patient/transaction).
- Strict fallback: if the guard fails, the system discards the narrative and emits a strict machine-readable template of the verified facts.
- Result: across every configuration in our runs the hallucination-free ratio is 100%.

8. How this benefits explainable AI (XAI)

The design turns an opaque neural prediction into a decision a human can audit. Each XAI property below is produced by a concrete mechanism in the code, not by post-hoc approximation.

XAI property	How the framework delivers it
Transparency	The symbolic class is an explicit sum of named rules with published thresholds, not a black box.
Faithfulness	The explanation is generated FROM the same facts that drove the decision, so it cannot diverge.
Traceability	Every case stores a full reasoning trace: features -> fired rules -> verification -> final decision.
Reliability / safety	Hard constraints can override unsafe neural outputs; low-confidence cases defer to logic.
No hallucination	The guard blocks invented rules/entities; fallback guarantees a safe output.
Human trust	Conflicts and overrides are surfaced as warnings instead of being hidden.
Privacy	Federated training means explanations are produced without ever centralising raw data.

- Neural-only vs neuro-symbolic: the neural model gives the accuracy; the symbolic layer gives the account of WHY, and corrects clear mistakes. The two together are more trustworthy than either alone.
- Because explanations are grounded in recognised thresholds (see NSF_Threshold_Evidence.pdf), a domain expert can check them against real guidelines.

9. Evidence from the actual runs

Domain	Strategy	Neuro-sym acc	Conflicts	Overrides	Halluc-free
healthcare	fedavg	88.0%	114	15	100%
healthcare	fedprox	89.3%	115	14	100%
healthcare	fedadagrad	90.0%	104	13	100%
banking	fedavg	99.0%	21	0	100%
banking	fedprox	98.8%	22	0	100%
banking	fedadagrad	99.0%	22	1	100%

Conflicts are cases where neural and symbolic layers disagreed (recorded for the explanation); overrides are

the subset where symbolic logic safely corrected the neural output. The hallucination-free ratio stays at 100%.

10. In one paragraph

A privacy-preserving federated MLP makes the prediction; a transparent symbolic engine verifies it against published, domain-grounded rules and can override unsafe outputs; and a fact-locked explainer - LLM-ready but not LLM-dependent - turns only the verified facts into an auditable explanation. The result is accurate, private, reliable and explainable by construction.